

LMD

Langage de Manipulation des Données

LMD

- En anglais **DML** (Data Manipulation Language)
- Langage sous-ensemble du **SQL** (Structured Query Language)
- **Permet la manipulation des données d'une Bdd à travers ses tables :**
 - Enregistrement de données dans les Tables d'une Bdd (ajout de tuples)
 - Modification des données contenues dans les Tables
 - Suppression de données contenues dans les tables (suppression de tuples)
 - Extraction et Exploitation des données (sélection, calcul et agrégation de tuples)

Les commandes de base

- **INSERT:** Insertion de tuples dans une table
- **UPDATE:** Modification de la valeur de colonnes des tuples d'une table
- **DELETE:** Suppression de tuples d'une table
- **SELECT:** Obtention d'un ensemble de tuples d'une table ou d'une vue

Insertion de données dans une table (1)

```
INSERT INTO <table> [(<colonne 1>, <colonne 2>, ...)]  
VALUES  (<expr ligne 1 colonne 1>, <expr ligne 1 colonne 2>, ...),  
        (<expr ligne 2 colonne 1>, <expr ligne 2 colonne 2>, ...),  
        ( ... );
```

- Ajout de tuples dans la **table** dont le nom est spécifié
- La **liste des colonnes** à alimenter pour chaque tuple inséré peut être spécifiée. Si cette liste n'est pas spécifiée alors l'insertion concernera **toutes les colonnes** de la table.
- Les **tuples à insérer** sont définis après le mot-clé **VALUE** . Ils sont constitués d'**ensemble d'expressions** dont l'évaluation fournit les **valeurs à affecter aux colonnes cibles**.
- Si une **colonne n'est pas spécifiée** ou qu'**aucune valeur** n'a été définie pour l'alimenter, elle prendra comme valeur **NULL** ou bien la **valeur par défaut** qui a été définie pour cette colonne à la création de la table.

Insertion de données dans une table ⁽²⁾

- Ajout de données dans une table à partir d'une « Requête »

```
INSERT INTO <table> [(<colonne 1>, <colonne 2>, ...)]  
SELECT ... ;
```

- Les **tuples à insérer** sont les résultats obtenus à partir d'un **SELECT**.
- **SELECT** est une instruction qui permet d'extraire un ensemble de tuples d'une ou plusieurs Tables: c'est la « Requête »
- Une variante de l'ajout à partir d'une « Requête » avec création de la table destination.

```
SELECT <colonne1, colonne2>  
INTO <tableDestination>  
FROM <TableOrigine>
```

Insertion de données dans une table ⁽³⁾

- Ajout de données dans une table à partir d'un fichier

```
COPY <table> FROM <fichier> [WITH (FORMAT text|csv|binary, ... )]
```

le fichier doit être lisible par le serveur

- Insertion de 2 types d'avions dans la table appareil :

```
INSERT INTO appareil (codetype, nbplace, design)
VALUES ('A50', 50, 'Arbus ARB50'),
       ('C22', 7, 'Cessna Mini')
```

codetype [PK] character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini

La mise à jour des données d'une table ⁽¹⁾

- Modification de la valeur de certaines colonnes des tuples d'une table

```
UPDATE <table>  
SET <colonne 1> = <expression 1>,  
    <colonne 2> = <expression 2>,  
    ...
```

- La **valeur** de chacune des **colonnes** spécifiées est remplacée par le résultat de l'**évaluation de l'expression** associée.

- Cette instruction s'applique à tous les tuples de la table. Il est possible d'ajouter une restriction pour choisir les tuples à impacter: **ceux pour lesquels le prédicat est vrai**.

```
UPDATE <table>  
SET <colonne 1> = <expression 1>,  
    <colonne 2> = <expression 2>,  
    ...  
WHERE <prédicat>
```

La mise à jour des données d'une table ⁽²⁾

- Modification du nombre de places et de la désignation de tous les appareils:

```
UPDATE appareil SET  
nbplace = nbplace + 5,  
design = design || ' (+5)'
```

codetype [PK] character(3)	nbplace smallint	design character varying(50)
A50	55	Arbus ARB50 (+5)
C22	12	Cessna Mini (+5)

- Modification du nombre de places des appareils de moins de 20 places:

```
UPDATE appareil SET  
nbplace = nbplace + 1  
WHERE nbplace < 20
```

codetype [PK] character(3)	nbplace smallint	design character varying(50)
A50	55	Arbus ARB50 (+5)
C22	13	Cessna Mini (+5)

Suppression de données d'une table ⁽¹⁾

- Suppression de tous les tuples d'une table pour lesquels le prédicat est vrai

```
DELETE FROM <table>  
WHERE <prédicat>
```

- Suppression de « l'Arbus 50 » de la table appareil.

```
DELETE FROM appareil  
WHERE codetype = 'A50'
```

codetype [PK] character(3)	nbplace smallint	design character varying(50)
C22	13	Cessna Mini (+5)

Extraction et Exploitation des données

- La **BDD relationnelle** comporte des **Tables** contenant des informations
- La normalisation du schéma a conduit à **répartir l'information** originale en **plusieurs tables**
- L'exploitation des données de la BDD consiste à utiliser ses tables pour:
 - **Retrouver et reconstituer les informations stockées** (la totalité ou certaines selon des critères)
 - **Agréger** l'ensemble des informations (somme, moyenne, maximum, minimum etc.)
- Les tables ou relations constituent un modèle relationnel exploitable en utilisant l'algèbre relationnelle

Algèbre relationnelle

- **Définition:** formalisme mettant à disposition un certain nombre d'opérateurs applicables sur des relations et combinables dans le but d'obtenir une relation résultat.
- Le **résultat** de toute opération **est une relation**
- **Propriétés:**
 - **La fermeture:** le résultat de l'application d'un opérateur sur une ou plusieurs relations est une relation pouvant être utilisée comme opérande d'une autre opération relationnelle.
 - **Les opérateurs sont ensemblistes:** manipulation d'ensemble de tuple et non de tuple unitaire. (ensemble de tuples sans ordre, ni de doublon)

Opérateurs de l'Algèbre relationnelle

■ Opérateurs de base:

- Différence
- Union
- Sélection ou restriction
- Projection
- Produit cartésien

■ Opérateurs dérivés:

- Intersection
- Jointure
- Division

Classification des Opérateurs

■ Opérateurs Unaire:

- **Sélection** : extraction d'un sous-ensemble de tuple selon critères
- **Projection** : extraction d'un sous-ensemble de colonnes

■ Opérateurs Binaire:

■ à schéma identique :

- **Différence** : obtention d'une relation comportant les tuples d'une première relation privés de ceux présent pas dans une seconde relation.
- **Union** : obtention d'une relation comportant tous les tuples de deux relations (un tuple ne peut figurer qu'une et une seule fois)
- **Intersection** : obtention d'une relation comportant les tuples présent dans les deux relations.

■ à schéma différent :

- **Produit cartésien** : obtention d'une relation comportant un schémas comportant les schéma des deux relations participantes. A chaque tuple de la première relation est associé chaque tuple de la seconde. (toutes les combinaisons)
- **Jointure** : produit cartésien de deux relations limité aux tuples qui satisfont une expression logique.
- **Division** : obtention d'une relation comportant les tuples pour lesquels le produit cartésien avec la seconde relation donne des tuples contenus dans la première relation (expression d'un quantificateur universel).

Opérateurs Ensemblistes : Union

■ **Définition** : Opération portant sur deux relations **R1** et **R2** ayant le même schéma dont le résultat est une 3^{ème} relation **R3** constituée des n-uplets appartenant à **R1** ou bien à **R2** sans doublon.

■ **Notation** : $R1 \cup R2$

■ **Représentation graphique**: 

■ **Exemple**:

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini

U

codetype character(3)	nbplace smallint	design character varying(50)
C22	7	Cessna Mini
747	150	Boing 747

=

codetype character(3)	nbplace smallint	design character varying(50)
747	150	Boing 747
C22	7	Cessna Mini
A50	50	Arbus ARB50

Opérateurs Ensemblistes : Différence

■ **Définition** : Opération portant sur deux relations **R1** et **R2** ayant le même schéma dont le résultat est une 3^{ème} relation **R3** constituée des **n-uplets appartenant à R1 et pas à R2**.

■ **Notation** : $R1 - R2$

■ **Représentation graphique**:



■ **Exemple**:

codetype	nbplace	design
character(3)	smallint	character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini

=

codetype	nbplace	design
character(3)	smallint	character varying(50)
C22	7	Cessna Mini
747	150	Boing 747

=

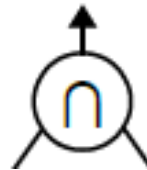
codetype	nbplace	design
character(3)	smallint	character varying(50)
A50	50	Arbus ARB50

Opérateurs Ensemblistes : Intersection

■ **Définition** : Opération portant sur deux relations **R1** et **R2** ayant le même schéma dont le résultat est une 3^{ème} relation **R3** constituée des **n-uplets appartenant à R1 et à R2**.

■ **Notation** : $R1 \cap R2$

■ **Représentation graphique**:



■ **Exemple**:

codetype	nbplace	design
character(3)	smallint	character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini

$\cap$

codetype	nbplace	design
character(3)	smallint	character varying(50)
C22	7	Cessna Mini
747	150	Boing 747

=

codetype	nbplace	design
character(3)	smallint	character varying(50)
C22	7	Cessna Mini

Opérateurs Ensemblistes : Division

■ **Définition** : Opération portant sur deux relations **R1** et **R2** avec le schéma de **R2** inclus dans celui de **R1** dont le résultat est une 3^{ème} relation **R3** constituée des **n-uplets appartenant à R1 qui associés à tous les tuples de R2 se retrouvent dans R1**.

■ **Notation** : $R1 \div R2$

■ **Représentation graphique**:



■ **Exemple**:

codetype	nbplace	design
character(3)	smallint	character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747



design
character varying(50)
Arbus ARB50



codetype	nbplace
character(3)	smallint
A50	50

Opérateurs Ensemblistes : Produit Cartésien

■ **Définition** : Opération portant sur deux relations **R1** et **R2** dont le résultat est une 3^{ème} relation **R3** constituée des **n-uplets** correspondants à toutes les combinaisons des tuples des relations **R1** et **R2**.

■ **Notation** : $R1 \times R2$

■ **Représentation graphique**:



■ **Exemple**:

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

X

nuavion [PK] character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22

=

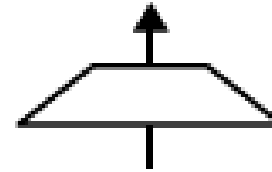
codetype character(3)	nbplace smallint	design character varying(50)	nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
A50	50	Arbus ARB50	0001	1995	Avion 1	1000	A50
A50	50	Arbus ARB50	0002	1997	Avion 2	980	C22
C22	7	Cessna Mini	0001	1995	Avion 1	1000	A50
C22	7	Cessna Mini	0002	1997	Avion 2	980	C22
747	150	Boing 747	0001	1995	Avion 1	1000	A50
747	150	Boing 747	0002	1997	Avion 2	980	C22

Opérateur unaire de Projection

■ **Définition** : Opération portant sur une relations **R** consistant à créer une relation **R'** ne comportant que les attributs **A₁, A₂, ..., A_n**.

■ **Notation** : $\pi (A_1, A_2, \dots, A_n) R$

■ **Représentation graphique**:



■ **Exemple**:

$\pi (\text{codetype}, \text{design})$

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

=

codetype character(3)	design character varying(50)
A50	Arbus ARB50
C22	Cessna Mini
747	Boing 747

Opérateur unaire de Sélection

■ **Définition** : Opération portant sur une relations **R** consistant à créer une relation **R'** comportant uniquement les tuples de **R** qui satisfont l'expression logique **E**.

■ **Notation** : $\sigma(E)R$

■ **Représentation graphique**:



■ **Exemple**:

$\sigma(\text{nbplace} < 100)$

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

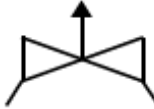
=

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini

Opérateur binaire de Jointure

■ **Définition** : Opération portant sur deux relations **R1** et **R2** dont le résultat est une 3^{ème} relation **R3** constituée des combinaisons des tuples de R1 et R2 qui **satisfont une expression logique E**.

■ **Notation** : $R1 \bowtie_E R2$ (équivalent à $\sigma(E)R1 \times R2$)

■ **Représentation graphique**: 

■ **Exemple**:

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22

$\bowtie_{\text{avion.typeapareil} = \text{appareil.codetype}}$

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

=

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)	codetype character(3)	nbplace smallint	design character varying(50)
0001	1995	Avion 1	1000	A50	A50	50	Arbus ARB50
0002	1997	Avion 2	980	C22	C22	7	Cessna Mini
0003	1991	Avion 3	5000	C22	C22	7	Cessna Mini

Opérateur binaire de semi-Jointure

■ **Définition** : Opération de jointure sur deux relations **R1** et **R2** favorisant une relation dont tous les tuples seront retenus que l'expression logique **E** soit satisfaite ou non.

■ **Notation** : $R1 \bowtie_E R2$

■ **Représentation graphique**: 

■ **Exemple**:

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeappareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22

$\bowtie_{\text{avion.typeappareil} = \text{appareil.codetype}}$

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50

=

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeappareil character(3)	codetype character(3)	nbplace smallint	design character varying(50)
0001	1995	Avion 1	1000	A50	A50	50	Arbus ARB50
0002	1997	Avion 2	980	C22			
0003	1991	Avion 3	5000	C22			

Le langage SQL

- **SQL: Structured Query Language** (Query : requête, demande)
- SQL permet d'exprimer des **requêtes basées sur l'algèbre relationnelle**
- Il fournit des **fonctions d'agrégat** : Somme, Moyenne, Min, Max
- **C'est le langage des SGBD Relationnels**
 - LDD
 - LMD
- **Normalisation:** par l'ANSI puis l'ISO
 - **SQL-86:** base
 - **SQL-89:** ajout de l'intégrité référentielle
 - **SQL-92:** norme de référence (spécification de types, requêtes imbriquées...)
 - **SQL-3: (1999)** ajout extensions objets

Requête SQL – Clauses de base

- **SELECT** : projection
- **FROM** : spécification des tables sur lesquelles porte la requête
- **WHERE** : conditions utilisées pour réaliser les sélections et restrictions, parfois la jointure.
- Une requête SQL commence par un **SELECT**

Requête SQL – Projection ⁽¹⁾

■ **SELECT** : permet de spécifier les colonnes à sélectionner dans la relation résultat.

```
-- Projection
SELECT codetype, nbplace, design
FROM appareil;

-- SELECT * --> tous les attributs sont retenus
SELECT *
FROM appareil;
```

■ **SELECT DISTINCT** : impose l'unicité des tuples.

```
SELECT DISTINCT appareil, design
FROM appareil;
```

Requête SQL – Projection ⁽²⁾

- **SELECT** : permet de spécifier des littéraux et des expressions calculées.
- Les **colonnes calculées** et les **colonnes déclarées** dans la clause **SELECT** peuvent être **nommées ou renommées**.
- Le type des **colonnes** résultats **peu** être modifié par CAST de la valeur. L'opérateur de CAST de Postgres est ::

```
-- Littéraux|
SELECT codetype, nbplace, design, 'AAA' literalAAA FROM appareil;

-- Utilisation Fonctions current_date, renommage en lesplaces et cast de la valeur en reel de nbplace
SELECT '#' || codetype || '#', current_date ,nbplace::real lesplaces, design FROM appareil;

--Expression calculée
SELECT codetype, nbplace, design, nbplace + 10 placePlus10 FROM appareil;
```

Requête SQL – Restriction / Sélection

■ **WHERE** : permet de spécifier les conditions à utiliser pour appliquer une restriction.

```
-- Liste des tuples de appareil  
-- avec la valeur de nbplace < à 100  
SELECT codetype, nbplace, design  
FROM appareil  
WHERE nbplace < 100;
```

■ Les conditions sont combinables avec des **ET** et des **OU**

```
-- appareil C22 et autres appareils  
-- avec nbplace entre 10 et 100  
SELECT codetype, nbplace, design  
FROM appareil  
WHERE (nbplace > 10 AND nbplace < 100)  
      OR codetype = 'C22';
```

Requête SQL – Produit cartésien

■ Produit cartésien de 2 tables.

X

codetype	nbplace	design
character(3)	smallint	character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

nuavion	anserv	nom	nbhvol	typeapareil
[PK] character(4)	smallint	character varying(50)	integer	character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22

=

codetype	nbplace	design	nuavion	anserv	nom	nbhvol	typeapareil
character(3)	smallint	character varying(50)	character(4)	smallint	character varying(50)	integer	character(3)
A50	50	Arbus ARB50	0001	1995	Avion 1	1000	A50
A50	50	Arbus ARB50	0002	1997	Avion 2	980	C22
C22	7	Cessna Mini	0001	1995	Avion 1	1000	A50
C22	7	Cessna Mini	0002	1997	Avion 2	980	C22
747	150	Boing 747	0001	1995	Avion 1	1000	A50
747	150	Boing 747	0002	1997	Avion 2	980	C22

■ Par défaut les tuples de toutes les tables de la clause **FROM** sont combinés

```
SELECT *  
FROM avion, appareil
```

Requête SQL – Jointure par Restriction

- Jointure par restriction du produit cartésien de 2 tables.

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22

⋈ avion.typeapareil = appareil.codetype

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

=

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)	codetype character(3)	nbplace smallint	design character varying(50)
0001	1995	Avion 1	1000	A50	A50	50	Arbus ARB50
0002	1997	Avion 2	980	C22	C22	7	Cessna Mini
0003	1991	Avion 3	5000	C22	C22	7	Cessna Mini

```
SELECT *  
FROM avion, appareil  
WHERE avion.typeapareil = appareil.codetype
```

Requête SQL – Jointure

- Jointure en utilisant l'instruction **JOIN** ou **INNER JOIN**.

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22

⋈ avion.typeapareil = appareil.codetype

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50
C22	7	Cessna Mini
747	150	Boing 747

=

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)	codetype character(3)	nbplace smallint	design character varying(50)
0001	1995	Avion 1	1000	A50	A50	50	Arbus ARB50
0002	1997	Avion 2	980	C22	C22	7	Cessna Mini
0003	1991	Avion 3	5000	C22	C22	7	Cessna Mini

```
SELECT *  
FROM avion  
INNER JOIN appareil ON avion.typeapareil = appareil.codetype
```

Requête SQL – Semi-Jointure

- Jointure en utilisant l’instruction **LEFT JOIN** ou **RIGHT JOIN**.

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22

⋈ avion.typeapareil = appareil.codetype

codetype character(3)	nbplace smallint	design character varying(50)
A50	50	Arbus ARB50

=

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeapareil character(3)	codetype character(3)	nbplace smallint	design character varying(50)
0001	1995	Avion 1	1000	A50	A50	50	Arbus ARB50
0002	1997	Avion 2	980	C22			
0003	1991	Avion 3	5000	C22			

2 Requêtes équivalentes:

```
SELECT *  
FROM avion  
LEFT JOIN appareil ON avion.typeapareil = appareil.codetype;  
  
SELECT *  
FROM appareil  
RIGHT JOIN avion ON appareil.codetype = avion.typeapareil;
```

Requête SQL – Les fonctions Agrégats ⁽¹⁾

- Les fonctions agrégats:

- **Min**: valeur minimum
- **Max**: valeur maximum
- **Avg**: moyenne
- **Count**: compte du nombre de valeurs
- **Sum**: somme

- Les fonctions agrégats **s'appliquent** sur des **attributs**

`MIN(nbplace), SUM(nbplace), AVG(nbplace) etc.`

- Agrégation de **tous les tuples** ou bien **des regroupements** de tuples par valeurs d'un ensemble d'attributs

Requête SQL – Les fonctions Agrégats ⁽²⁾

- Requête avec des agrégats sur la totalité des tuples:

```
-- Somme du nombre de places de tous les appareils  
SELECT SUM(nbplace)  
FROM appareil;
```

- Requête avec des agrégats sur des regroupements de tuple:
GROUP BY attribut1, attribut2, ..., attribut N

```
-- Nombre d'avions par type d'appareil  
SELECT typeappareil, count(*)  
FROM avion  
GROUP BY typeappareil
```

typeappareil character(3)	nombre bigint
747	1
A50	2
C22	2

nuavion character(4)	anserv smallint	nom character varying(50)	nbhvol integer	typeappareil character(3)
0001	1995	Avion 1	1000	A50
0002	1997	Avion 2	980	C22
0003	1991	Avion 3	5000	C22
0004	2010	Avion 4	250	747
0005	2006	Avion 5	620	A50

Requête SQL – Les sous-requêtes ⁽¹⁾

- Utilisation de **sous-requêtes** dans la **clause de sélection**:

- **IN** : la valeur à gauche du **IN** doit appartenir aux résultats de la sous-requête

```
-- Avion dont la désignation du type appareil contient 'RB'  
SELECT * FROM avion WHERE typeappareil  
IN (SELECT codetype FROM appareil WHERE design LIKE '%RB%')
```

- **EXISTS** : la sous-requête doit retourner au moins un tuple

```
SELECT * FROM avion  
WHERE EXISTS (SELECT 1 FROM appareil WHERE design LIKE '%RB%'  
              AND avion.typeappareil = appareil.codetype)
```

- **ANY** : au moins une des valeurs retournées par la sous-requête doivent vérifier l'expression

```
SELECT * FROM appareil  
WHERE appareil.design = ANY (SELECT design FROM appareil a );
```

- **ALL** : toutes les valeurs retournées par la sous-requête doivent vérifier l'expression

```
SELECT * FROM appareil  
WHERE appareil.design = ALL (SELECT design FROM appareil a );
```

Requête SQL – Les sous-requêtes (2)

- Les sous-requête dans la clause **FROM** : les tables utilisées par la requête peuvent être le résultat d'une requête

```
SELECT * FROM (  
    SELECT avion.*, appareil.design |  
    FROM avion JOIN appareil on avion.typeappareil = appareil.codetype  
), avion_appareil
```